

Expense Splitter — Full-Stack Technical Deep Dive

Complete Comprehensive Architecture, System Design & Interview Preparation Master Guide

Role / Context: Full-Stack Developer Interview	Tech Stack: MERN (MongoDB, Express v5, React 19, Node v22)
Auth Model: JWT in HTTP-Only SameSite Cookies	Deployment: Monorepo Architecture (Vite + Node)

1. What the Project Is

Expense Splitter (repository name *Expense-Tracker*, client app *tour_expense*) is a production-grade, full-stack web application engineered to solve the complex financial coordination problems that arise when groups of people share expenses during trips, events, or shared living arrangements.

Core Problem Solved:

When groups travel or live together, different members pay out-of-pocket for various shared costs (hotels, meals, gas, entertainment). Manually calculating individual shares, managing unequal splits, tracking who owes whom, and settling up leads to mathematical errors, awkward financial disputes, and friction. Expense Splitter automates group financial ledgers, supports custom unequal splitting, provides pairwise debt generation, and delivers an admin-managed settlement workflow.

Real-World Use Case Scenario:

Imagine 4 friends (Alice, Bob, Charlie, and David) go on a 3-day road trip:

- **Hotel:** Alice pays \$12,000 for all 4 people (Equal split = \$3,000 each).
- **Dinner:** Bob pays \$3,000, but David only had drinks (Unequal custom split: Alice \$1,000, Bob \$1,000, Charlie \$800, David \$200).
- **Gas:** Charlie pays \$2,000 for Alice, Bob, and Charlie only.

Without an app, calculating net balances requires complex pairwise balance reconciliations. With Expense Splitter, each member views their exact live ledger (**You Owe** vs **You Get** vs **Net Balance**), and the group admin can mark debts as settled in one click.

2. Architecture & System Flow

System Architecture Overview:

The system follows a classic decoupled 3-Tier Monorepo Architecture: Single-Page Application (SPA) Client, RESTful API Server, and Document Database.

Client Tier (Frontend)	Server Tier (Backend API)	Database Tier
React 19 + Vite Tailwind CSS v4 Framer Motion React Router v7 React Toastify	Node.js v22 + Express v5 JWT Cookie Auth Bcrypt Password Hashing Cors with Credentials Mongoose ODM v9	MongoDB Atlas / Local Collections: • <i>users</i> • <i>groups</i> • <i>expenses</i>

Authentication & Data Request Lifecycle:

- Client Request:** React frontend makes HTTP requests using standard fetch() with credentials: 'include'.
- CORS & Middleware Interception:** Express backend runs CORS check against FRONTEND_URL, parses cookies via cookie-parser.
- Token Validation (validateUser):** Middleware extracts JWT token from req.cookies.token, verifies signature with JWT_KEY, fetches User document (excluding password), and attaches object to req.user.
- Business Logic Execution:** Route controller computes business logic (e.g., split amounts, invoice ledger, arrayFilter updates).
- Database Persistence:** Mongoose executes MongoDB queries with population or atomic array updates.
- Client State Hydration:** React receives JSON response and updates state via Framer Motion reactive UI.

3. Important Components & File Breakdown

File Path & Module	Role & Purpose	Key Technology / Concepts Used
backend/server.js	Application entry point; mounts routes, configures CORS with credentials, express.json, and cookie-parser.	Express v5, CORS whitelist, cookie-parser
backend/config/db.js	Establishes async connection to MongoDB database via Mongoose.	Mongoose v9, process.env.MONGO_URL
backend/models/User.js	User schema defining profile attributes, unique lowercase username, email, and array of Group ObjectIds.	Mongoose Schema, ObjectId refs to 'Group'
backend/models/Group.js	Group schema containing groupName, description, admin ref, members refs, expenses refs, and totalGroupExpense.	ObjectId refs to 'User' and 'Expense'
backend/models/Expense.js	Expense schema storing spentFor, totalExpense, paidBy, splitType, groupID, and participants array with sharedAmount & isSettled.	Sub-document arrays, Boolean settlement flags
backend/middleware/validateUser.js	JWT Authentication guard. Extracts token from HttpOnly cookie, verifies JWT, attaches req.user.	jsonwebtoken (jwt.verify), HTTP Cookies
backend/controllers/authController.js	Handles registration (username auto-generation, bcrypt hashing), login (token generation, cookie setting), and logout.	bcrypt (salt 10), res.cookie(httpOnly, sameSite)

backend/routes/authRoutes.js	Auth routing + profile fetch/update endpoints and regex-based user search API.	MongoDB \$regex search, \$options: 'i', \$ne filter
backend/routes/groupsRoutes.js	Group endpoints: group creation (linked to members), group deletion (cascading delete), group fetch with multi-level population.	Promise.all, Mongoose populate, \$pull
backend/routes/expenseRoutes.js	Core financial engine: add expense (equal/custom split), getInvoice (debt ledger), settle (arrayFilters), edit/delete/bulkDelete.	MongoDB arrayFilters (\$elem[]), deleteMany
frontend/src/App.jsx	Frontend Router configuration with dynamic parameter routes and ProtectedRoute wrappers.	React Router v7 (BrowserRouter, Routes, Route)
src/components/common/ProtectedRoute.jsx	Higher-Order Component guarding routes by pingin /validateUser before rendering children.	React useEffect, useState, Navigate
src/pages/GroupDetails.jsx	Primary workspace managing tabs (Summary, Add Expense, History, Settlement) and member sidebar.	Framer Motion AnimatePresence, tab state
src/components/expenses/AddExpense.jsx	Dual-mode form supporting Equal split calculation and Custom manual per-participant amounts.	Dynamic controlled inputs, custom validation
src/components/settlements/Invoice.jsx	Summary dashboard displaying 'You Owe', 'You Get', 'Net Balance' and detailed Giving/Getting lists.	Object.entries, array reductions
src/components/settlements/FinalSettlement.jsx	Calculates net pairwise balances between members and renders admin 'Settle Up' action trigger.	Conditional balance evaluation, Admin guard

4. End-to-End Working (Step-by-Step)

Flow 1: User Registration & Authentication Setup

1. User submits form on /Register with name, username, email, and password.
2. Request arrives at POST /api/auth/register.
3. Backend checks if email or username already exists in User collection. If username is missing, it auto-generates one from the email prefix.
4. Password is hashed using bcrypt.hash(password, 10).
5. User document is saved. JWT token is signed with payload { email, _id }.
6. Token is set in response cookie: httpOnly: true, sameSite: 'lax'/none', maxAge: 7 days.
7. User is redirected to Dashboard.

Flow 2: Group Creation & Bi-directional Referencing

1. User navigates to CreateGroup page, enters Group Name, Description, and searches members via UserSearch.
2. Submit pings POST /user/group/:userID/create_group.
3. Server maps member ObjectIds, prepends the creator's ID as admin.
4. Group document is created.
5. Server runs Promise.all() across all member IDs, updating each user's document by pushing the new createdGroup._id into user.groups array.

Flow 3: Expense Logging & Division Math

1. In GroupDetails, user submits 'Add Expense' form (e.g. Total ■3,000, Paid by Alice, Participants: Alice, Bob, Charlie).
2. If splitType === 'equal': Backend computes sharedAmount = totalExpense / participants.length (■1,000 each).
3. If splitType === 'custom': Backend reads customAmounts[userID] for each participant.
4. Expense document is created in MongoDB.
5. Server updates Group document: group.totalGroupExpense += totalExpense and group.expenses.push(expense._id).
6. *Error Rollback*: If updating the group fails, a manual rollback executes: await expenseModel.findByIdAndDelete(expense._id).

Flow 4: Invoice Ledger & Admin Settlement

1. Component pings GET /user/expense/:userID/:groupID/getInvoice.
2. Server finds expenses where logged-in user is either paidBy OR in participants.
3. Server iterates expenses, skipping settled items (!isSettled):
 - If User paid: accumulates non-settled participant shares into getExpenses.
 - If User participated & didn't pay: accumulates share owed to payer into giveExpenses.
4. **Admin Settle Up**: Admin clicks 'Settle Up' for Member X -> pings POST /user/expense/:groupID/settle/:withUserID.
5. Server uses MongoDB updateMany with arrayFilters to mark matching participant sub-documents as isSettled: true.

5. Database Schemas & API Reference

MongoDB Models & Relationships:

```
// User Schema { name: String, username: { type: String, unique: true }, email: String, password: String, groups: [{ type: ObjectId, ref: 'Group' }] } // Group Schema { groupName: String, description: String, admin: { type: ObjectId, ref: 'User' }, members: [{ type: ObjectId, ref: 'User' }], expenses: [{ type: ObjectId, ref: 'Expense' }], totalGroupExpense: Number } // Expense Schema { spentFor: String, totalExpense: Number, paidBy: { type: ObjectId, ref: 'User' }, splitType: String, groupID: { type: ObjectId, ref: 'Group' }, participants: [{ userID: { type: ObjectId, ref: 'User' }, sharedAmount: Number, isSettled: { type: Boolean, default: false } } ] }
```

Complete REST API Reference Matrix:

HTTP Method & Endpoint	Auth	Description & Operation	Key Request / Response Payload
POST /api/auth/register	No	Registers user, hashes password, generates JWT cookie.	Req: { name, username, email, password } Res: { auth: true, user }
POST /api/auth/login	No	Authenticates user credentials, sets HttpOnly JWT cookie.	Req: { email, password } Res: { auth: true, user }
GET /api/auth/validateUser	Yes	Validates JWT token for ProtectedRoute verification.	Res: { success: true } or 401 Auth False
GET /api/auth/getUser	Yes	Fetches logged-in user profile populated with groups.	Res: { success: true, user } (password omitted)
GET /api/auth/searchUsers	Yes	Regex search for users by name/email/username excluding current user.	Query: ?q=john Res: { success: true, users: [] }
POST /user/group/:userID/create_group	Yes	Creates group, assigns admin, updates all member user models.	Req: { groupName, description, members: [] }
GET /user/group/:groupID/getGroup	Public/User	Fetches group populated with members, admin, and expenses.	Res: { success: true, group: { ... } }
DELETE /user/group/:groupID/deleteGroup	Yes (Admin)	Deletes group, pulls ref from users, deletes group expenses.	Res: { success: true, message }
POST /user/expense/:groupID/addExpense	Yes	Calculates equal/custom split, creates expense, updates group total.	Req: { spentFor, paidBy, totalExpense, splitType, participants, customAmounts }
GET /user/expense/:userID/:groupID/getInvoice	Public/User	Computes non-settled giving/getting ledger for user.	Res: { getExpenses: {}, giveExpenses: {} }
POST /user/expense/:groupID/settle/:withUserID	Yes (Admin)	Executes atomic MongoDB arrayFilters update marking debts settled.	Res: { success: true, message: 'Settled up successfully!' }
POST /user/expense/bulkDelete	Yes (Admin)	Deletes array of selected expenses and updates group totals.	Req: { expenseIDs: [...] }

6. Key Technical Concepts (Interview Friendly)

1. JWT Authentication in HttpOnly Cookies vs LocalStorage

Simple Concept: LocalStorage is readable by JavaScript, making tokens vulnerable to Cross-Site Scripting (XSS) attacks. By setting JWTs in HttpOnly cookies, the browser automatically attaches the cookie to requests, but client-side JavaScript cannot access it via document.cookie.

2. Mongoose Schema Population (.populate())

Simple Concept: MongoDB is a non-relational document database without native SQL JOINS. Mongoose .populate() simulates JOIN behavior at the application layer by reading stored ObjectIds and automatically firing a second query to hydrate referenced documents.

3. MongoDB Positional Operators & Array Filters (\$[elem])

Simple Concept: When updating an object buried inside an array in MongoDB (e.g. setting isSettled: true for a specific participant in an expense), arrayFilters allows precise targeting of elements matching a filter condition without rewriting the entire array.

```
// MongoDB arrayFilter snippet from expenseRoutes.js await expenseModel.updateMany( { groupId, paidBy: userID, "participants.userID": withUserID }, { $set: { "participants.$[elem].isSettled": true } }, { arrayFilters: [{ "elem.userID": withUserID }] } );
```

4. Bi-directional Schema Referencing

Simple Concept: In our database, Group stores an array of member User IDs, and User stores an array of Group IDs. This dual-linking provides O(1) direct query access from both directions at the cost of keeping references synchronized.

7. Design Decisions & Architectural Trade-offs

Design Decision	Chosen Approach	Alternative Approach	Trade-off Rationale & Evaluation
Token Storage	HttpOnly, SameSite Cookie	LocalStorage / SessionStorage	HttpOnly cookies block XSS token theft. Requires CORS credentials: true configuration.
Relational Fetching	Mongoose .populate()	MongoDB \$lookup Aggregation	Population is cleaner in JavaScript code. Aggregation pipeline is significantly faster for large datasets.
Debt Ledger Model	Pairwise Ledger Generation	Greedy Min-Cash-Flow Graph Reduction	Pairwise keeps transparent expense line-items per person. Graph reduction minimizes total transactions (e.g. 3-way split).
Multi-Doc Updates	Manual try-catch rollback	MongoDB ACID Transactions (Sessions)	Manual rollback works on standalone MongoDB. ACID transactions require MongoDB Replica Sets.

8. Security, Scalability & Error Handling

Security Mechanisms Implemented:

- **Password Security:** Passwords hashed via bcrypt with 10 salt rounds before database insertion.
- **Token Protection:** JWT stored in HttpOnly cookies prevents client JavaScript access.
- **Route Guards:** validateUser middleware protects routes and sets req.user.
- **Admin Authorization:** Group deletion, member addition, expense edits, and settlement restricted to group.admin.

Potential Security Gaps & Missing Features (Honest Interview Points):

- **No Rate Limiting:** Endpoints like /login and /register lack express-rate-limit, exposing them to brute-force attacks.
- **Input Schema Validation:** Lacks validation libraries like Zod or Joi to validate body payload types.
- **CSRF Vulnerability:** When using sameSite: 'none' across domains, explicit CSRF token verification (e.g. Double Submit Cookie) should be added.

Scalability Bottlenecks & Optimization Strategies:

1. **Unindexed MongoDB Queries:** Fields like `groupId` and `paidBy` in `Expense` need database compound indexes (`{ groupId: 1, paidBy: 1 }`).
2. **Unbounded Array Growth:** Storing all expense `ObjectIds` inside `Group.expenses` can exceed MongoDB's 16MB document limit if a group has thousands of expenses. Fix: Query expenses via `Expense.find({ groupId })` with pagination.
3. **Floating-Point Rounding:** In JS, $100 / 3 = 33.333333333333336$. Division floating-point errors must be handled using integer cents (e.g., storing `100.00` as `10000` paise).

9. Interview Preparation & Question Bank

Q1: How does authentication work in your application?

Answer: I implemented stateless authentication using JWT stored in `HttpOnly SameSite` cookies. When a user logs in, backend verifies password hash with `bcrypt`, signs a JWT containing email and user ID, and sends it back in an `HttpOnly` cookie. On subsequent requests, `validateUser` middleware extracts and verifies the cookie token before granting access.

Q2: Why did you store JWT in `HttpOnly` cookies instead of `LocalStorage`?

Answer: Storing JWT in `LocalStorage` makes it accessible to any JavaScript code running on the page, rendering it vulnerable to XSS attacks. `HttpOnly` cookies cannot be accessed via `document.cookie` by client scripts, isolating the token from malicious scripts.

Q3: How do you handle unequal expense splitting?

Answer: In the expense controller, I check `splitType`. If `'equal'`, it divides `totalExpense` by participant count. If `'custom'`, it reads `customAmounts` map sent from frontend, validates that every participant has an assigned share, and constructs participants array with individual `sharedAmount` properties.

Q4: How does the Settlement feature work under the hood?

Answer: Settlement updates the `isSettled` boolean flag inside the expense participants array. The backend uses `Mongoose updateMany` with positional arrayFilters: `{ $set: { 'participants.$[elem].isSettled': true } }` where `elem.userID` matches the target member.

Q5: What happens if an error occurs while updating group expenses after creating an expense?

Answer: I implemented a manual rollback pattern in try-catch block. If group update fails after expense creation, the backend catches the error, calls `expenseModel.findByIdAndDelete(expense._id)` to delete orphaned expense, and returns a 500 error.

Q6: How would you optimize performance if a group has 50,000 expenses?

Answer: First, remove expenses array from Group document to avoid 16MB document limit. Second, add compound database index on Expense collection { groupId: 1, addedAt: -1 }. Third, implement pagination on history tab and convert Mongoose populate to MongoDB Aggregation pipeline.

Q7: How did you implement real-time user search when adding group members?

Answer: On the frontend, UserSearch component captures input query and calls GET /api/auth/searchUsers?q=searchterm. Backend executes regex query userModel.find({ \$or: [{username: regex}, {email: regex}, {name: regex}], _id: { \$ne: req.user._id } }) limited to 10 results.

Q8: What is the optimal algorithm for settling group debts (Splitwise algorithm)?

Answer: The Greedy Min-Cash-Flow graph algorithm. Instead of pair-by-pair payments, compute net balance for each person. Separate people into debtors and creditors, then iteratively match largest debtor with largest creditor. This reduces transaction count to at most N-1.

Q9: What cross-domain issues did you face between Vite frontend and Express backend?

Answer: Setting credentials: true on frontend fetch requires backend CORS to specify explicit origin process.env.FRONTEND_URL (not wildcard '*'). Additionally, cookie requires sameSite: 'none' and secure: true in production HTTPS environments.

Q10: How would you scale this application for 100,000 active users?

Answer: 1) Deploy stateless Express API across multiple auto-scaling containers behind a Load Balancer. 2) Implement Redis cache for active user profiles and group meta. 3) Upgrade MongoDB to a Sharded Cluster sharded by groupId.

10. Final Cheat Sheet & 1-Minute Pitch

The 1-Minute Project Elevator Pitch (Memorize This!):

"Expense Splitter is a full-stack MERN application designed to automate group expense tracking and debt settlements, similar to Splitwise. I built the backend with Node.js, Express, and MongoDB using Mongoose, and the frontend with React 19, Tailwind CSS, and Framer Motion.

Key features include dual-mode expense division (equal and custom unequal splits), dynamic search for group members, and a real-time debt ledger that calculates 'You Owe', 'You Get', and 'Net Balance'. For security, I implemented JWT authentication stored in HTTP-Only cookies with bcrypt password hashing. On the database side, I utilized MongoDB arrayFilters for atomic nested array updates and bi-directional schema references. If I had more time, I would replace pairwise settlement with the Greedy Min-Cash-Flow graph algorithm and add MongoDB ACID transactions."

Top 5 Golden Technical Takeaways:

1. **Auth:** JWT in HttpOnly cookies = XSS mitigation.
2. **DB Updates:** MongoDB arrayFilters (\$[elem]) = atomic nested array updates.
3. **Splitting Engine:** Supports both equal division and custom participant maps.
4. **CORS:** credentials: 'include' requires exact origin matching, not wildcard.
5. **System Improvement:** Upgrade debt calculation from Pairwise to Greedy Min-Cash-Flow Graph Algorithm.